

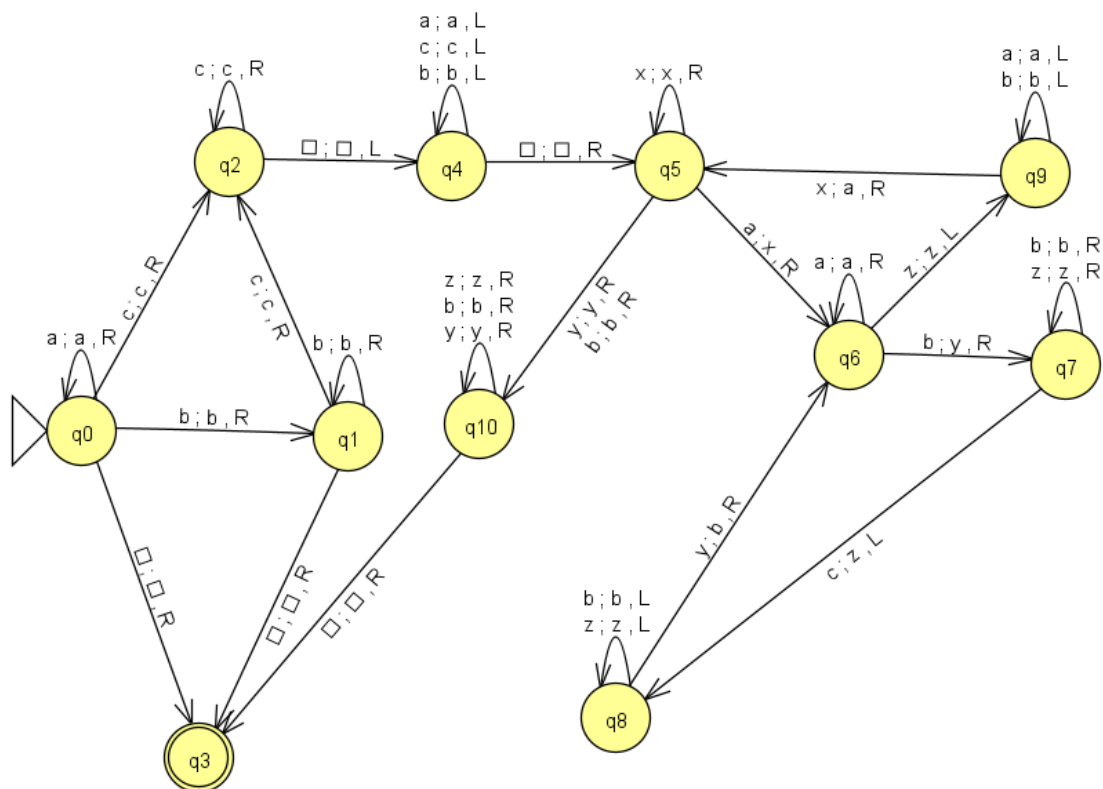
Teoria da Computação - Relatório I

Maria Júlia de Melo Honorato (23200368), Thayssa Yasmin Alcantara de Godoi (19201619)

1. $L = \{a^i b^j c^k \mid i, j, k \in \mathbb{N} \text{ e } i \times j = k\}$ em uma Máquina de Turing com fita única.

M = “com uma string de entrada w:

1. Varre o input da esquerda para a direita para determinar se pertence à $a^*b^*c^*$.
 - a. Se tiver símbolos fora de ordem, rejeita.
 - b. Se encontrar uma string vazia, ou composta somente por a's, ou composta somente por b's, aceita.
2. Retorna o cabeçote para a extremidade esquerda da fita.
3. Marca o primeiro 'a' e transforma em 'x'. Move o cabeçote para o próximo 'b'. Alterna entre 'b' e 'c', e para cada 'b' marcado como 'y', marca um 'c' como 'z'. Restaura os b's marcados com 'y' depois que marcou o 'c' correspondente.
 - a. Se todos os c's foram marcados, mas ainda tem b's não usado, rejeita.
4. Repete o estágio 3 se existirem mais a's.
5. Se passou por todos os a's, verifica se todos os c's já foram marcados.
 - a. Se não tiver nenhum 'c' sobrando, aceita.
 - b. Caso contrário, rejeita.”

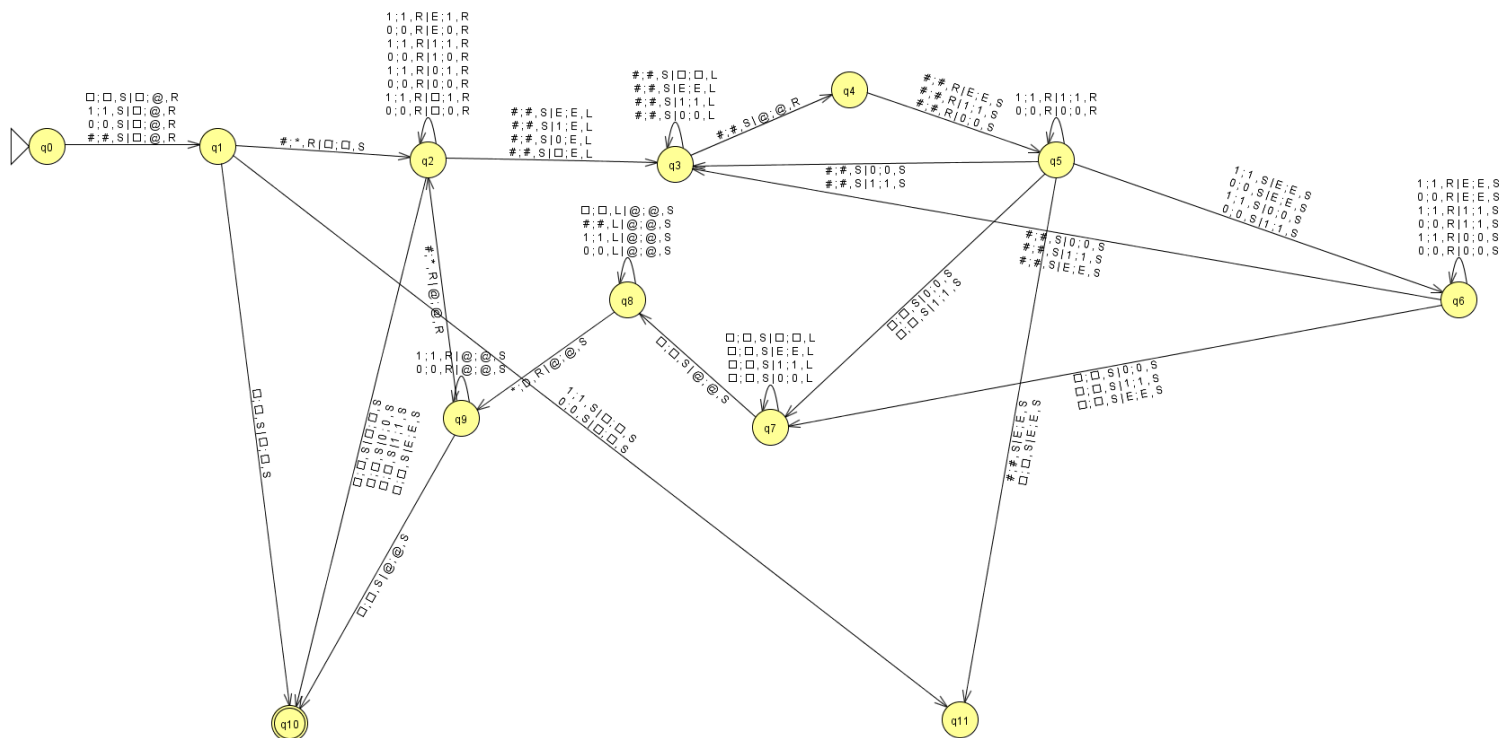


	Accept
a	Accept
aaa	Accept
b	Accept
bb	Accept
bbb	Accept
c	Reject
cba	Reject
aabac	Reject
abbc	Reject
ba	Reject
abcb	Reject
aabbbcccc	Accept
aaaabbbcccccccccccc	Accept
aaabccc	Accept
abccc	Reject
aabcc	Accept
aaabcc	Reject
aaabcccc	Reject
abbcccc	Accept

2. $L = \{ \#x_1\#x_2\#x_3\ldots\#x_n \mid x_i \in 0,1^* \text{ e } x_i \neq x_j \text{ para cada } i \neq j \}$ em uma Máquina de Turing Multifita.

M = "com uma string de entrada w fornecida integralmente na primeira fita:

1. Confere se w começa com '#'. Se w for vazia, aceita (não há blocos a comparar). Se o primeiro símbolo não for '#', rejeita, pois toda palavra da linguagem começa com '#'.
2. Marca o '#' do primeiro bloco como bloco corrente e copia o conteúdo desse bloco (os símbolos de $\{0,1\}$ que vão até o próximo '#' ou até o fim da entrada) para a segunda fita, marcando o fim da cópia.
3. Percorre, da esquerda para a direita, cada bloco que está à direita do bloco corrente e compara, símbolo a símbolo, o conteúdo desse bloco (na primeira fita) com a cópia guardada na segunda fita. Se os dois forem idênticos, mesmo comprimento e mesmos símbolos, inclusive quando ambos são vazios, rejeita, pois encontrou $x_i = x_j$. b. Caso contrário, segue para o próximo bloco à direita.
4. Terminada a varredura à direita, marca o bloco corrente como já processado, avança para o bloco seguinte, torna-o o novo bloco corrente e copia o seu conteúdo para a segunda fita (apagando a cópia anterior). Repete o estágio 3.
5. Quando o bloco corrente for o último (não houver mais nenhum bloco à direita), aceita, pois todos os pares de blocos foram comparados e nenhum par é igual."



No Multiple Run foram usadas as entradas abaixo:

Entradas válidas (Accept):

- # apenas o bloco vazio ($n = 1$)
- #0 um único bloco "0"
- #1 um único bloco "1"
- #01 um único bloco "01"

Entradas inválidas (Reject):

- 0#1 não começa com '#'
- ## dois blocos vazios iguais ($x_1 = x_2 = \varepsilon$)

Input 1	Input 2	Result
#		Accept
#0		Accept
#1		Accept
#01		Accept
0#1		Reject
##		Accept

Como a Multiple Run do JFLAP não trata bem máquinas multifita em entradas longas (o simulador gera uma configuração por fita a cada passo e não as funde, o que faz o número de configurações crescer exponencialmente), as entradas mais longas foram verificadas no modo passo a passo (Input → Step...), que deduplica as configurações e executa normalmente:

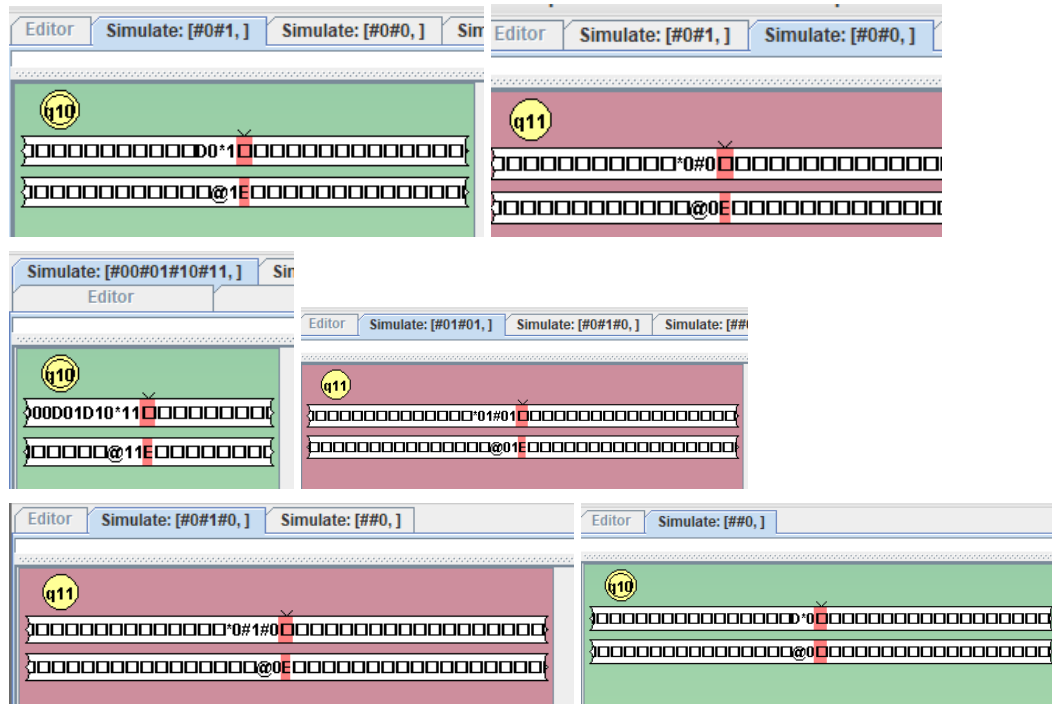
Entradas válidas (Accept):

#0#1 blocos "0" e "1", distintos
 #00#01#10#11 quatro blocos, todos distintos
 ##0

Entradas inválidas (Reject):

#0#0 blocos "0" e "0", iguais ($x_1 = x_2$)
 #01#01 blocos "01" e "01", iguais
 #0#1#0

Em todos os casos a máquina produziu o resultado esperado: aceitou exatamente as cadeias em que todos os blocos são distintos e rejeitou as que possuem algum par de blocos iguais ou que não começam com '#'.
 #'

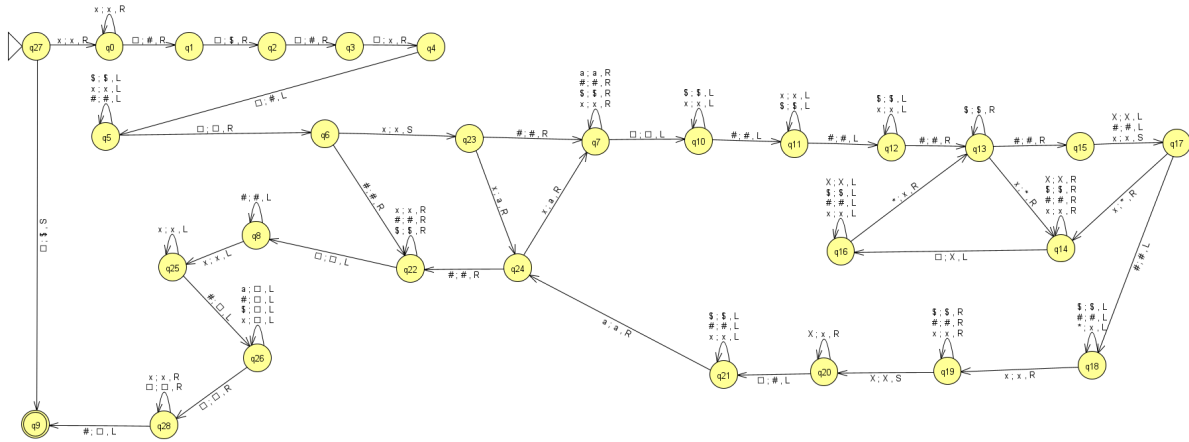


3.a) Série de Fibonacci. A máquina recebe como entrada uma sequência de símbolos que representa n (representação unária - como X ou a). Ao término, deve constar na fita uma sequência de símbolos que indica o valor do n -ésimo termo, ou seja $Fibonacci(n)$. Faça com que a máquina, em seus primeiros passos de computação grave as sementes 0 e 1 e proceda o cálculo iterativo da série até n .

M = “com uma string de entrada w no formato unário representando o índice do número de Fibonacci desejado:

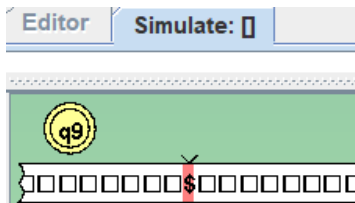
1. Varre o input da esquerda para a direita até encontrar o fim da palavra.
 - a. Se for vazia, marca \$ para representar '0' e aceita.
 - b. Se não for vazia, adiciona 0 (com '\$') e 1 (unário) separados com #, e retorna o cabeçote para a extremidade esquerda.
2. Marca o 'x' do primeiro termo como 'a' na primeira iteração, e marca o próximo para o processamento iterativo. O cabeçote se move até o penúltimo termo (termos divididos por #).
3. Na primeira iteração o penúltimo será \$, e o termo vai ser ignorado, e a iteração será só sobre o último (nesse caso, será 1). Depois da primeira, a máquina itera sobre o penúltimo e último termo, adicionando um 'X' para cada 'x' dos termos no final da fita. Os x's são marcados com * e restaurados durante a varredura.
4. Depois da varredura acabar, transforma X's em x's e adiciona um # no final para separar o novo termo e volta para a esquerda.
 - a. Se no primeiro termo ainda tiverem x's não marcados, repete o estágio 2.

- b. Se só tiverem a's, transforma todos os caracteres do penúltimo # para esquerda em vazio, depois caminha até o último #, o apaga e aponta para o último caractere da resposta final, e aceita.

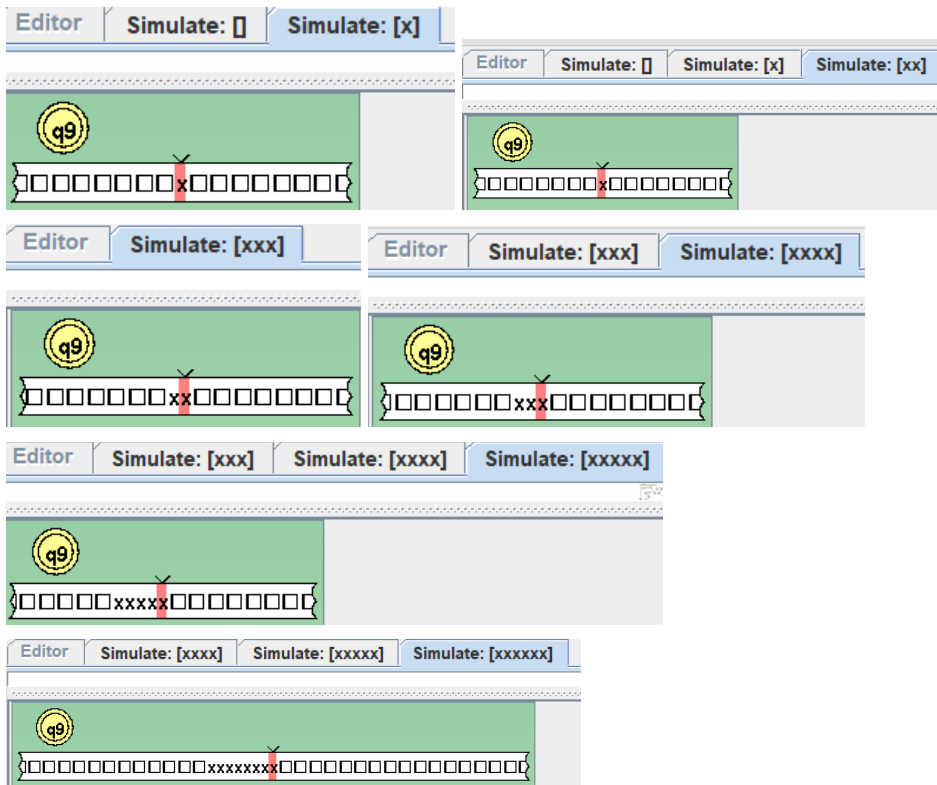


Foi feito considerando a existência de $\text{Fibonacci}(0) = 0$, representação com 'x' e foi usado '\$' para representar o zero. A fita final fica somente com a representação do valor do n-ésimo termo, com o cabeçote apontando para o último caractere.

$\text{Fibonacci}() = \$$



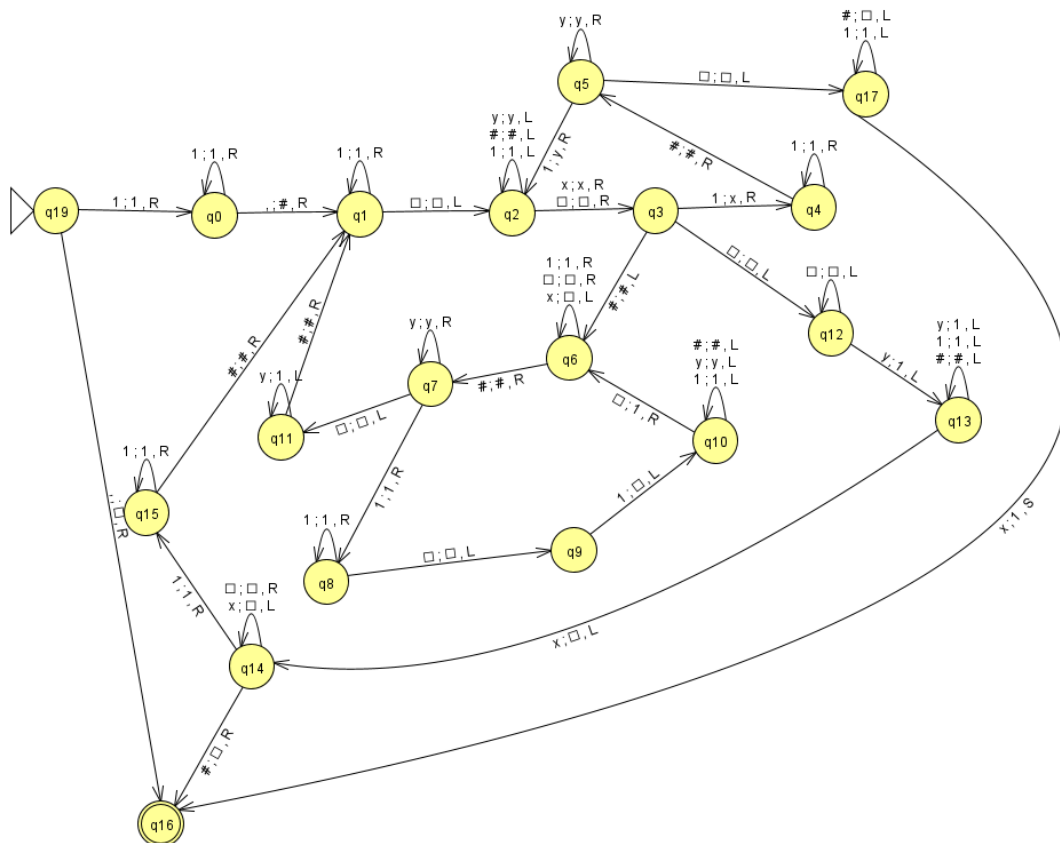
$\text{Fibonacci}(x) = x$ até $\text{Fibonacci}(xxxxxx) = xxxxxxxx$



3.b) Algoritmo de Euclides para o Máximo Divisor Comum. A máquina recebe como entrada uma sequência de símbolos representando n e m em representação unária. Ao término, a fita deve conter o $MDC(n, m)$.

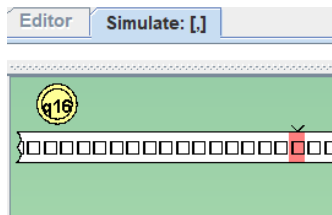
M = “com uma string de entrada w no formato unário representando o índice do número de Fibonacci desejado:

1. A máquina varre a entrada para verificar casos base.
 - a. Se não tiver nada antes do # e tiver símbolos na direita, limpa a fita e aceita, mantendo o valor da direita. O mesmo acontece se só tiver símbolos na esquerda.
 - b. Se ambos os lados estiverem vazios, limpa a fita e aceita.
2. A máquina começa o loop principal buscando o primeiro 1 do lado esquerdo e um correspondente do lado direito, marcando com x e y , respectivamente. O cabeçote volta para a esquerda e repete o processo.
 - a. Se no final da varredura sobrar 1's do lado esquerdo, $n > m$, e a máquina limpa os x 's e restaura os y 's e reinicia o estágio 2.
 - b. Se no final da varredura sobrar 1's do lado direito, $m > n$, a máquina limpa os x 's, e para cada '1' que sobrou do lado direito, apaga o original e insere um '1' do lado esquerdo. Restaura os y 's e reinicia o estágio 2.
3. Se na varredura $n = m$, o loop principal é interrompido e faz a limpeza final.
 - a. Restaura os y 's, limpa os x 's e o #, e a resposta final é a quantidade de 1's que sobraram na fita.

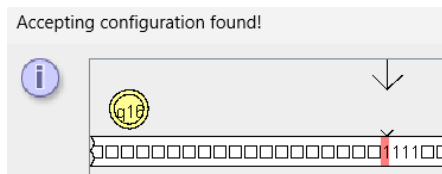


Feito presumindo existência de $MDC(0,0) = 0$.

- $MDC(0,0) = 0$:



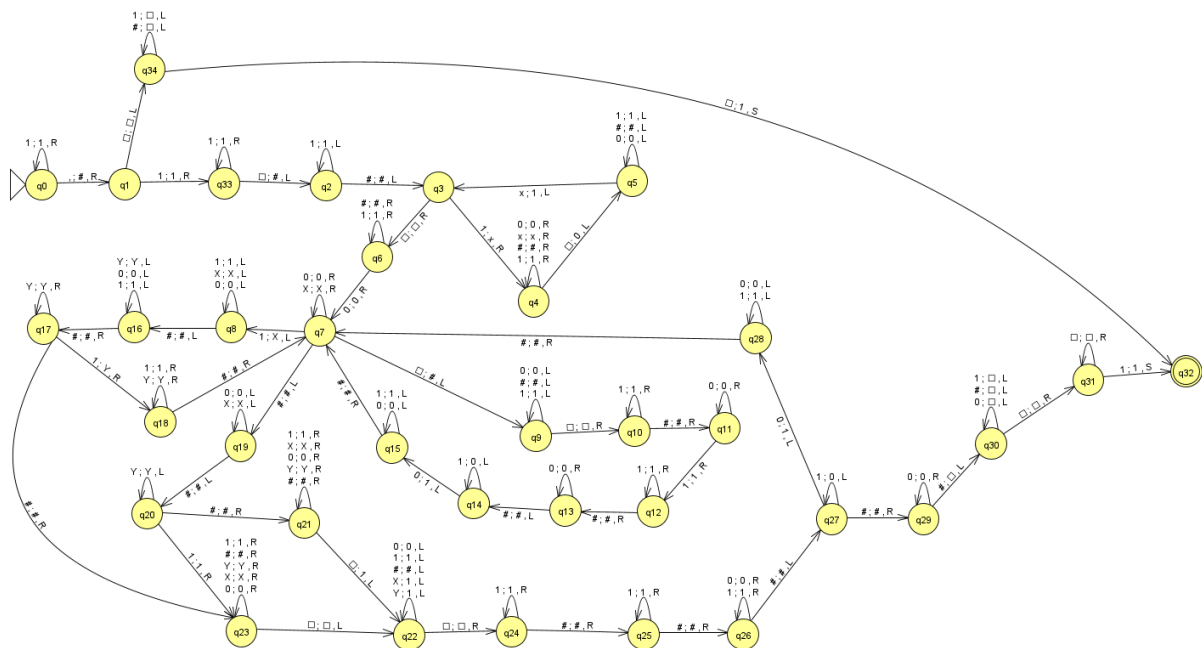
- $MDC(31,20) = 4$:



3.c) Triângulo de Pascal. A máquina recebe como entrada um par (linha, coluna) e retorna o elemento correspondente no triângulo.

M = “com uma string de entrada *w* no formato *unário,unário* representando linha *n* e coluna *k*:

1. Varre o input da esquerda para a direita até encontrar a vírgula e identificar o tamanho de *n*.
 - a. Se a entrada estiver somente com ‘,’ a máquina entende como a coordenada (0,0), escreve ‘1’ e aceita.
 - b. Se não tiver nenhum número depois da vírgula (coluna = 0), escreve ‘1’ e aceita.
2. É adicionado uma # após a entrada, e depois dela uma string auxiliar com a mesma quantidade de zeros que o tamanho da linha *n*. Isso acontece marcando um ‘1’ de *n* como ‘x’ e colocando um zero depois do último #. 1’s são restaurados durante a varredura. Um novo # é adicionado no final e o cabeçote retorna para a extremidade esquerda da fita.
3. Cabeçote procura um ‘0’ na string auxiliar, da direita para a esquerda, e muda o primeiro que achar para ‘1’.
4. Compara a quantidade de 1’s na string auxiliar com a quantidade de *k*, marcando com ‘X’ na auxiliar e com ‘Y’ na *k*. Continua esse processo enquanto tiver 1’s para marcar na string auxiliar com par em *k*.
 - a. Se não sobrar nenhum ‘1’ em nenhuma das duas strings, vai até o final da fita e adiciona um ‘1’ na zona de resposta.
 - b. Se sobraem 1’s, a máquina ignora a zona de resposta, e transforma os X’s e Y’s de volta em ‘1’.
5. A máquina ‘soma 1’ na string auxiliar (soma binária), transforma os 1’s da direita em 0’s até encontrar o primeiro ‘0’, o transformando em ‘1’. Se o carácter da direita for ‘0’, se transforma em ‘1’.
 - a. Se a soma binária funcionar, a máquina retorna o cabeçote para esquerda e repete a partir do estágio 4.
6. A soma binária eventualmente transforma a string auxiliar em somente 0’s. A máquina apaga tudo do último # para a esquerda. Depois anda para direita até achar o primeiro ‘1’.
 - a. Se em algum momento a zona de resposta foi usada, eventualmente o cabeçote vai parar no início da resposta final, e a máquina aceita.
 - b. Se coluna > linha (com linha != 0), a zona de resposta nunca foi usada e a máquina fica num loop andando para a direita.”



Foi utilizado a combinatória, para evitar contas matemáticas muito complexas de outras fórmulas comuns para encontrar um elemento dentro do triângulo de pascal. Gera sequencialmente todas as combinações binárias do tamanho da linha, e valida somente os resultados com a mesma quantidade de 1's do tamanho da coluna.

Entrada: '111,1111' (3, 5).

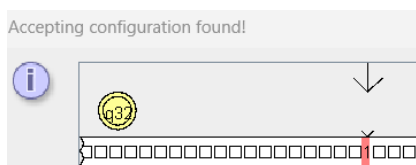
Foi considerado a contagem desde (0, 0), nesse caso, se quiser inserir 0, é só não digitar nada.

Entradas com 0: '111' (inválido, mas representa (0,3))

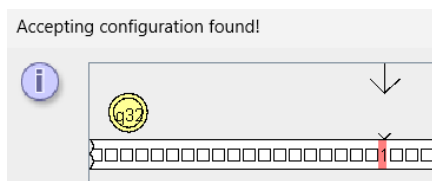
'1' (1, 0)

' ' (0, 0)

- linha = coluna:



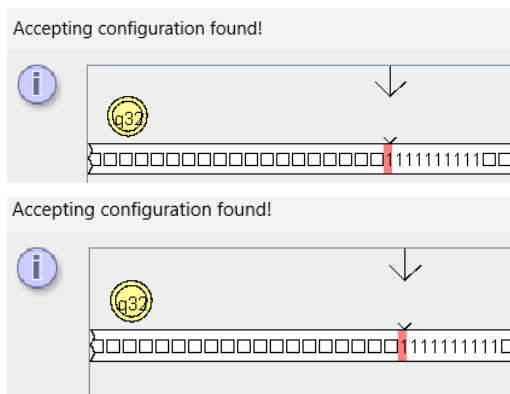
- coluna = 0:



- coluna > linha: entrada inválida, ficará num loop (rejeita se linha = 0)

11,1111 Cancelled

- 1111,11 - (5, 2) e 1111,111 (5, 3) = 10 -> simetria:



- 1111,11 (4, 2):

